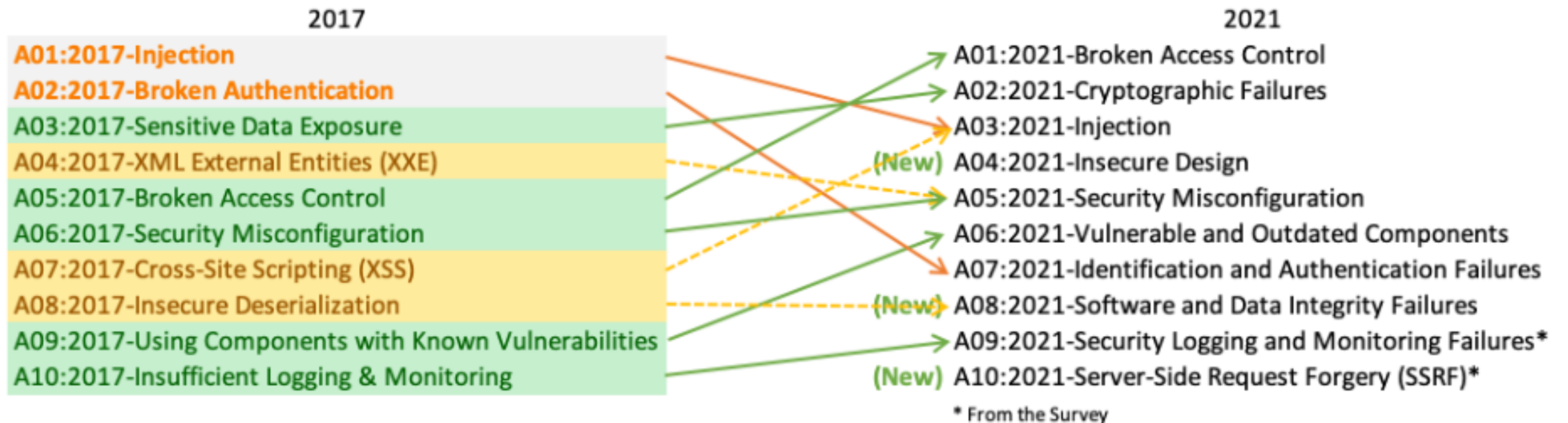




Introduction

OWASP Top 10



Software Security – What to Consider?

- The fundamental security problem with web applications — that all user input is untrusted — gives rise to a number of security mechanisms that applications use to defend themselves against attack.
- Virtually all applications employ mechanisms that are conceptually similar, although the details of the design and the effectiveness of the implementation vary greatly.
- The defense mechanisms employed by web applications comprise the following core elements:
 1. Handling user input to the application's functions to prevent malformed input from causing undesirable behavior
 2. Handling user access to the application's data and functionality to prevent users from gaining unauthorized access
 3. Handling attackers to ensure that the application behaves appropriately when being directly targeted, taking suitable defensive and offensive measures to frustrate the attacker
 4. Managing the application itself by enabling administrators to monitor its activities and configure its functionality

1/ Handling User Input

- **All user input is untrusted.**
- A huge variety of attacks against web applications involve submitting unexpected input, crafted to cause behavior that was not intended by the application's designers.
- Correspondingly, a key requirement for an application's security defenses is that the application must handle user input in a **safe manner**.
- Input-based vulnerabilities can arise anywhere within an application's functionality, and in relation to practically every type of technology in common use.
- **"Input validation"** is often cited as the necessary defense against these attacks. However, no single protective mechanism can be employed everywhere, and defending against malicious input is often not as straightforward as it sounds.

SQL Injection

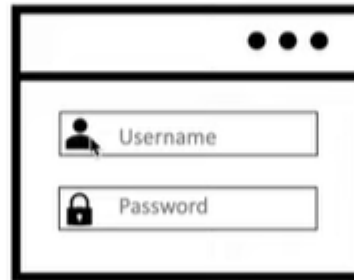


What is SQL Injection

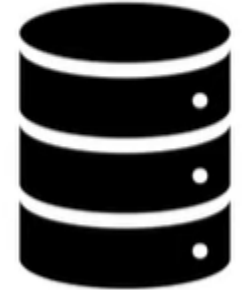
Attacker



Web Server

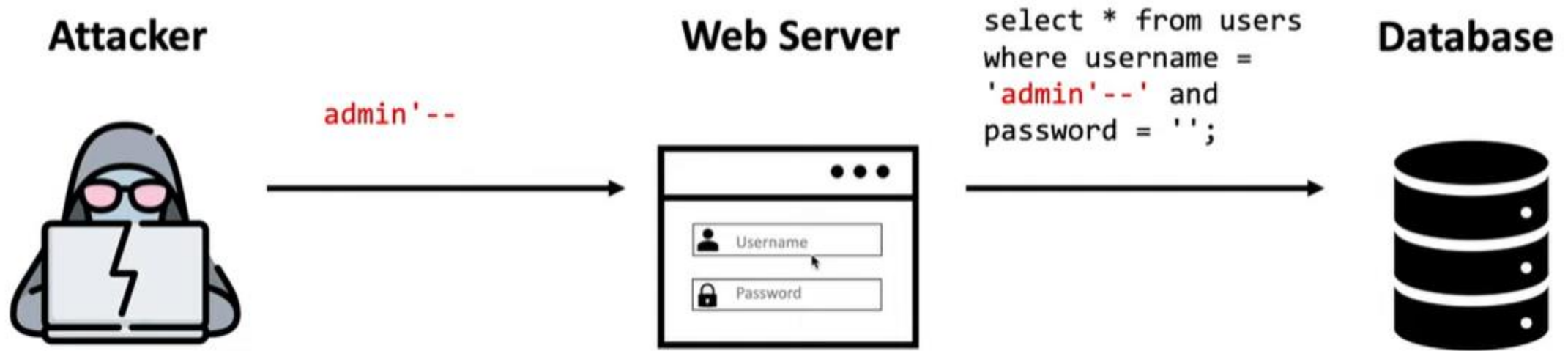


Database



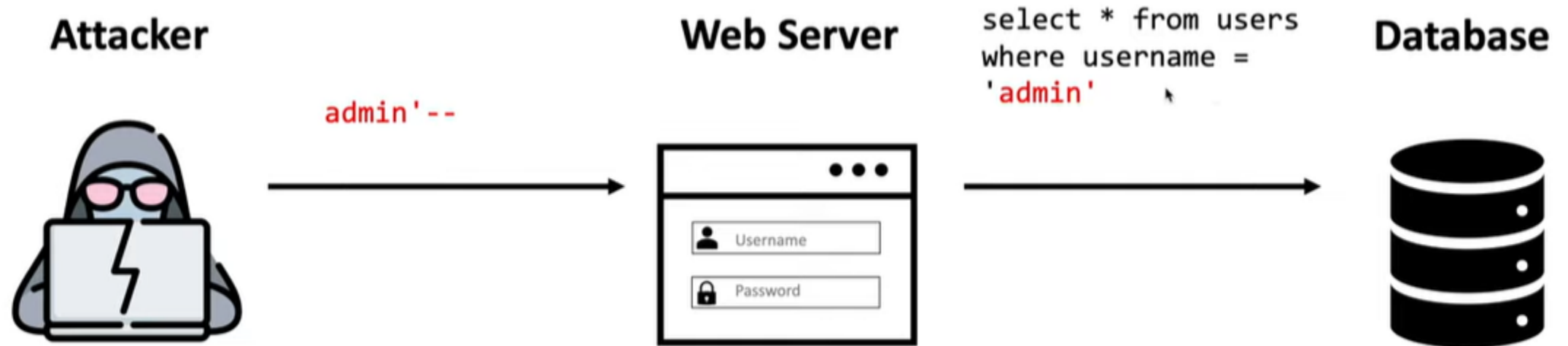


What is SQL Injection





What is SQL Injection

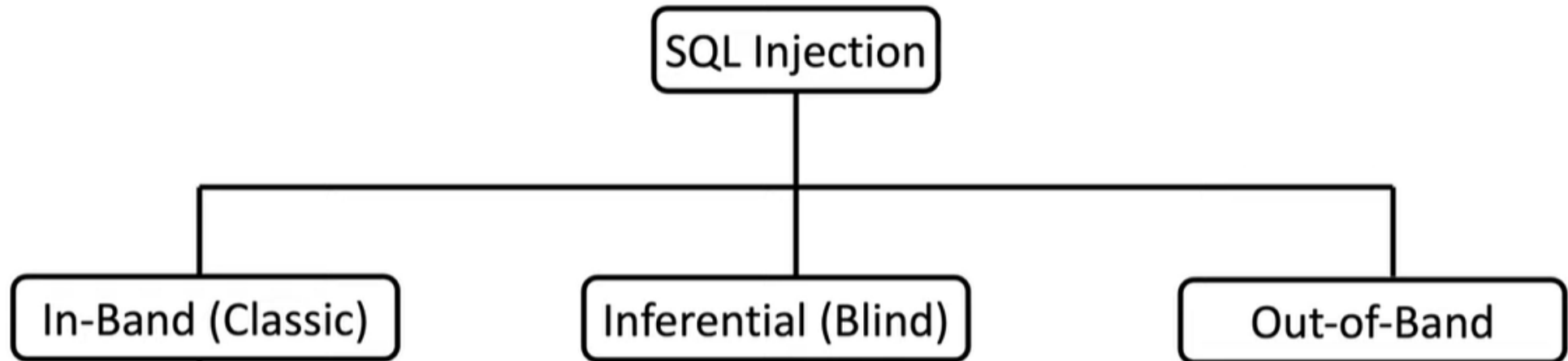


Causes of SQL Injection

- Attackers can use SQL injection on an application if it has dynamic database queries that use string concatenation and user supplied input. To avoid SQL injection flaws, developers need to:
 - Stop writing dynamic queries with string concatenation or
 - Prevent malicious SQL input from being included in executed queries.

```
String query = "SELECT account_balance FROM user_data WHERE user_name = "  
              + request.getParameter("customerName");  
try {  
    Statement statement = connection.createStatement( ... );  
    ResultSet results = statement.executeQuery( query );  
}
```

Types of SQL Injection



In Band SQL Injection

- Most SQL injection vulnerabilities occur within the WHERE clause of a SELECT query.
- However, SQL injection vulnerabilities can occur at any location within the query, and within different query types. Some other common locations where SQL injection arises are:
 - In UPDATE statements, within the updated values or the WHERE clause.
 - In INSERT statements, within the inserted values.
 - In SELECT statements, within the table or column name.
 - In SELECT statements, within the ORDER BY clause.

In Band SQL Injection - Retrieving hidden data

- Imagine a shopping application that displays products in different categories. When the user clicks on the Gifts category, their browser requests the URL:

<https://insecure-website.com/products?category=Gifts>

- This causes the application to make a SQL query to retrieve details of the relevant products from the database:

```
SELECT * FROM products WHERE category = 'Gifts' AND released = 1
```

This SQL query asks the database to return:

all details (*)
from the products table
where the category is Gifts
and released is 1.

In Band SQL Injection - Retrieving hidden data

The application doesn't implement any defenses against SQL injection attacks. This means an attacker can construct the following attack, for example:

<https://insecure-website.com/products?category=Gifts'-->

This results in the SQL query:

*SELECT * FROM products WHERE category = 'Gifts'--' AND released = 1*

Crucially, note that -- is a comment indicator in SQL. This means that the rest of the query is interpreted as a comment, effectively removing it. In this example, this means the query no longer includes AND released = 1. As a result, all products are displayed, including those that are not yet released.

In Band SQL Injection - Retrieving hidden data

You can use a similar attack to cause the application to display all the products in any category, including categories that they don't know about:

<https://insecure-website.com/products?category=Gifts'+OR+1=1-->

This results in the SQL query:

*SELECT * FROM products WHERE category = 'Gifts' OR 1=1--' AND released = 1*

The modified query returns all items where either the category is Gifts, or 1 is equal to 1. As 1=1 is always true, the query returns all items.

In Band SQL Injection - Subverting application logic

Imagine an application that lets users log in with a username and password. If a user submits the username wiener and the password bluecheese, the application checks the credentials by performing the following SQL query:

```
SELECT * FROM users WHERE username = 'wiener' AND password = 'bluecheese'
```

If the query returns the details of a user, then the login is successful. Otherwise, it is rejected.

In this case, an attacker can log in as any user without the need for a password. They can do this using the SQL comment sequence -- to remove the password check from the WHERE clause of the query. For example, submitting the username administrator'-- and a blank password results in the following query:

```
SELECT * FROM users WHERE username = 'administrator'--' AND password = ''
```

This query returns the user whose username is administrator and successfully logs the attacker in as that user.

In Band SQL Injection - UNION Keyword

The UNION keyword enables you to execute one or more additional SELECT queries and append the results to the original query. For example:

```
SELECT a, b FROM table1 UNION SELECT c, d FROM table2
```

This SQL query returns a single result set with two columns, containing values from columns a and b in table1 and columns c and d in table2.

For a UNION query to work, two key requirements must be met:

- **The individual queries must return the same number of columns.**
- **The data types in each column must be compatible between the individual queries.**

In Band SQL Injection - UNION Keyword

- When you use UNION in SQL, you are telling the database: “Take the results of Query A and stack them on top of the results of Query B.”
- For that to work: The database needs to know how to align the results column by column.
- If Query A has 3 columns and Query B has 2, SQL won’t know where to put the extra data.

SELECT id, name, email FROM users

UNION

SELECT username, password FROM admin; (x fails!)

In Band SQL Injection - UNION Keyword

- When you perform a SQL injection UNION attack, there are two effective methods to determine how many columns are being returned from the original query.
- One method involves injecting a series of ORDER BY clauses and incrementing the specified column index until an error occurs. For example, if the injection point is a quoted string within the WHERE clause of the original query, you would submit:
 - ' ORDER BY 1--
 - ' ORDER BY 2--
 - ' ORDER BY 3--
 - etc.
- This series of payloads modifies the original query to order the results by different columns in the result set. The column in an ORDER BY clause can be specified by its index, so you don't need to know the names of any columns. When the specified column index exceeds the number of actual columns in the result set, the database returns an error,

In Band SQL Injection - UNION Keyword

- The second method involves submitting a series of UNION SELECT payloads specifying a different number of null values:
- ' UNION SELECT NULL--
- ' UNION SELECT NULL,NULL--
- ' UNION SELECT NULL,NULL,NULL--
- etc.
- If the number of nulls does not match the number of columns, the database returns an error,

In Band SQL Injection - UNION Keyword

Why NULL?

NULL is type-flexible → it can fit into almost any column type (string, int, date, etc.). This makes it easier to test without guessing specific data types.

Imagine a vulnerable query:

```
SELECT id, name, email FROM users WHERE id = '1';
```

The attacker doesn't know it's 3 yet.

Step 1: Try with 1 column

The attacker injects:

```
SELECT id, name, email FROM users WHERE id = '1' UNION SELECT NULL— (fail)
```

```
SELECT id, name, email FROM users WHERE id = '1' ' UNION SELECT NULL,NULL,NULL—  
(success)
```

In Band SQL Injection - UNION Keyword

Once they know the column count, the attacker can start swapping NULLs with actual values from other tables.

For example:

<initial SQL query>' UNION SELECT username, password, email FROM admin--

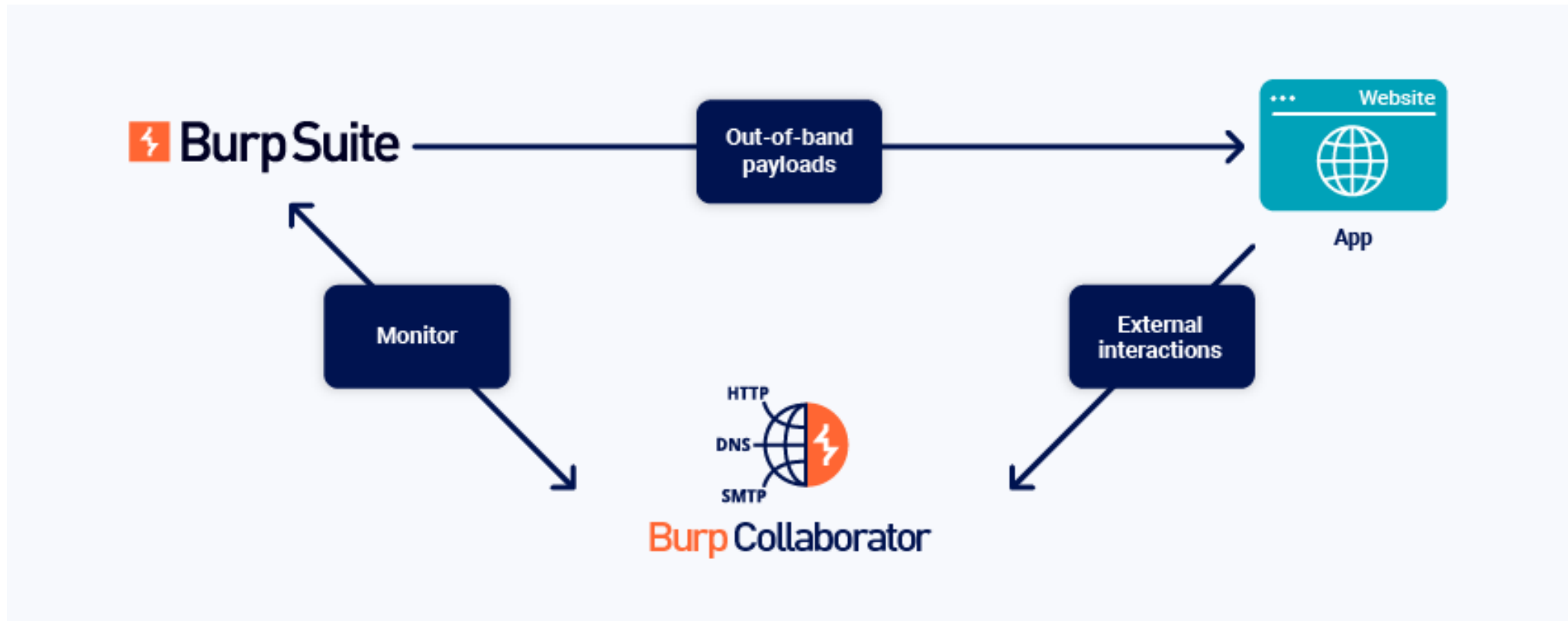
Out of Band SQL Injection

- An application might carry out the same SQL query as the previous example but do it asynchronously. The application continues processing the user's request in the original thread, and uses another thread to execute a SQL query using the tracking cookie. The query is still vulnerable to SQL injection, but none of the techniques described so far will work. The application's response doesn't depend on the query returning any data, a database error occurring, or on the time taken to execute the query.

Out of Band SQL Injection

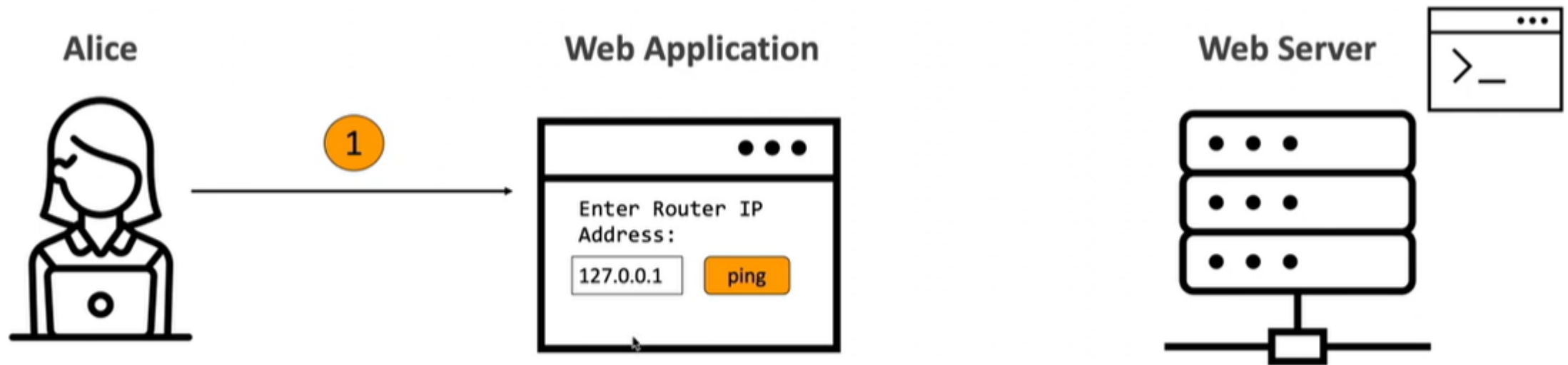
- In this situation, it is often possible to exploit the blind SQL injection vulnerability by triggering out-of-band network interactions to a system that you control. These can be triggered based on an injected condition to infer information one piece at a time. More usefully, data can be exfiltrated directly within the network interaction.
- A variety of network protocols can be used for this purpose, but typically the most effective is DNS (domain name service). Many production networks allow free egress of DNS queries, because they're essential for the normal operation of production systems.

Out of Band SQL Injection

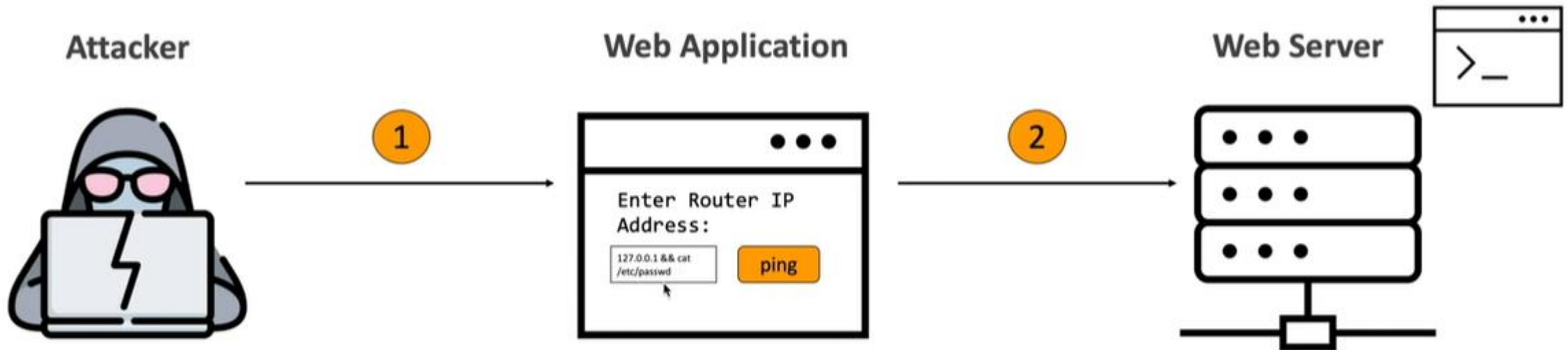


Command Injection

What is a command injection/OS injection



What is a command injection/OS injection



What is a command injection/OS injection

OS Command Injection is a vulnerability that consists of an attacker executing commands on the host operating system via a vulnerable application.

```
1 import java.io.IOException;
2 import javax.servlet.http.HttpServletRequest;
3 public void runUnsafe(HttpServletRequest request) throws IOException {
4     String cmd = request.getParameter("command");
5     String arg = request.getParameter("arg");
6     Runtime.getRuntime().exec(cmd+" "+arg);
7 }
```

Line #6 allows execution of arbitrary commands via client-side input.

What is a command injection/OS injection

1. In-band Command Injection

Consists of an attacker executing commands on the host operating system via a vulnerable application and receiving the response of the command in the application.

2. Blind Command Injection

Consists of an attacker executing commands on the host operating system via a vulnerable application that does not return the output from the command within its HTTP response.

In Band Command Injection

- a shopping application lets the user view whether an item is in stock in a particular store. This information is accessed via a URL:

`https://insecure-website.com/stockStatus?productID=381&storeID=29`

- To provide the stock information, the application must query various legacy systems. For historical reasons, the functionality is implemented by calling out to a shell command with the product and store IDs as arguments:

`stockreport.pl 381 29`

- This command outputs the stock status for the specified item, which is returned to the user.

In Band Command Injection

- The application implements no defenses against OS command injection, so an attacker can submit the following input to execute an arbitrary command:

& echo aiwefwlguh &

- If this input is submitted in the productId parameter, the command executed by the application is:

stockreport.pl & echo aiwefwlguh & 29

- The echo command causes the supplied string to be echoed in the output. This is a useful way to test for some types of OS command injection. The & character is a shell command separator. In this example, it causes three separate commands to execute, one after another. The output returned to the user is:

*Error - productId was not provided
aiwefwlguh
29: command not found*

- Shell metacharacters: `&`, `&&`, `|`, `||`, `;`, `\n`, ```, `$()`
- Concatenate another command

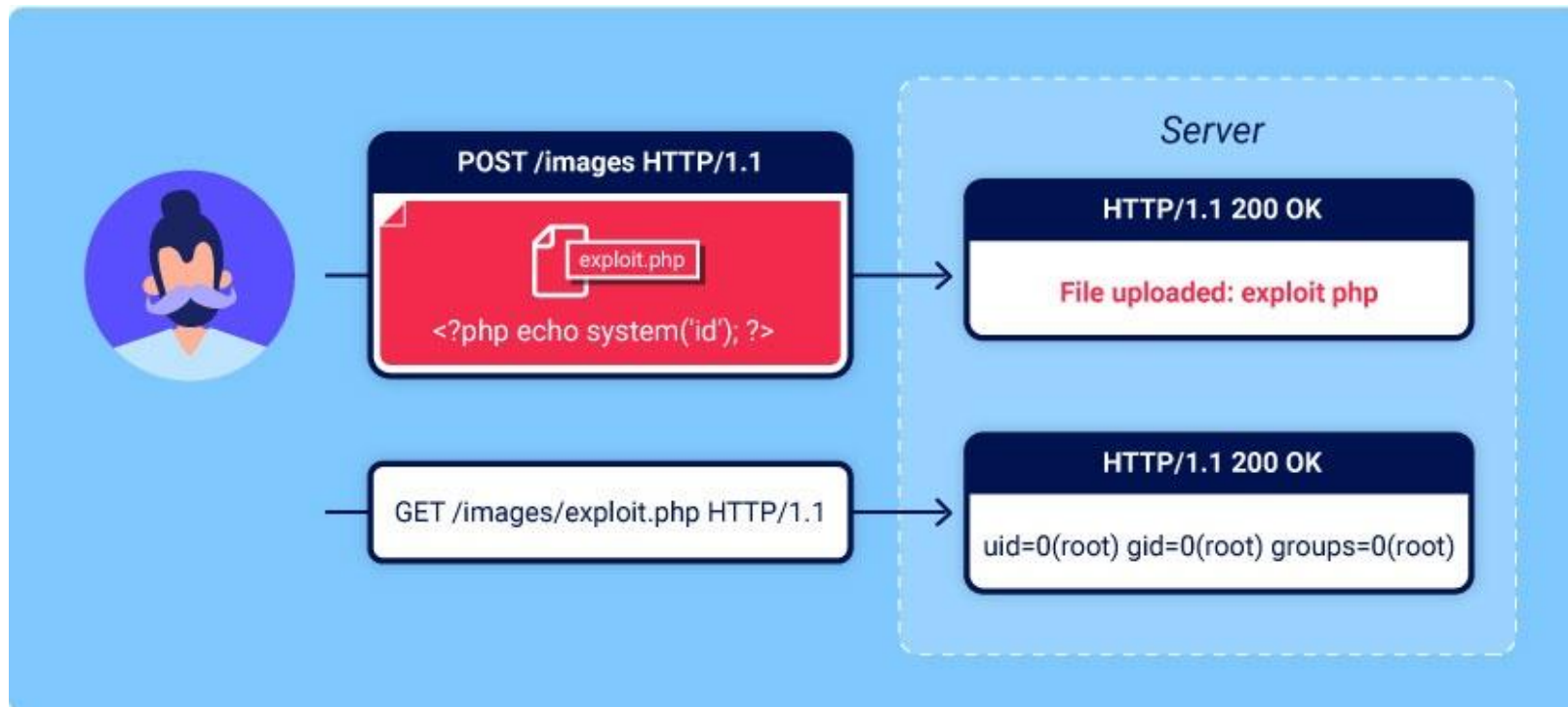
```
127.0.0.1 && cat /etc/passwd &
```

```
127.0.0.1 & cat /etc/passwd &
```

```
127.0.0.1 || cat /etc/passwd &
```


File upload vulnerabilities

- File upload vulnerabilities are when a web server allows users to upload files to its filesystem without sufficiently validating things like their name, type, contents, or size.
- Failing to properly enforce restrictions on these could mean that even a basic image upload function can be used to upload arbitrary and potentially dangerous files instead. This could even include server-side script files that enable remote code execution.



How do web servers handle requests for static files?

- The process for handling static files is still largely the same. The server parses the path in the request to identify the file extension. It then uses this to determine the type of the file being requested, typically by comparing it to a list of preconfigured mappings between extensions and MIME types.
 - If this file type is non-executable, such as an image or a static HTML page, the server may just send the file's contents to the client in an HTTP response.
 - If the file type is executable, such as a PHP file, and the server is configured to execute files of this type, it will assign variables based on the headers and parameters in the HTTP request before running the script. The resulting output may then be sent to the client in an HTTP response.
 - If the file type is executable, but the server is not configured to execute files of this type, it will generally respond with an error.

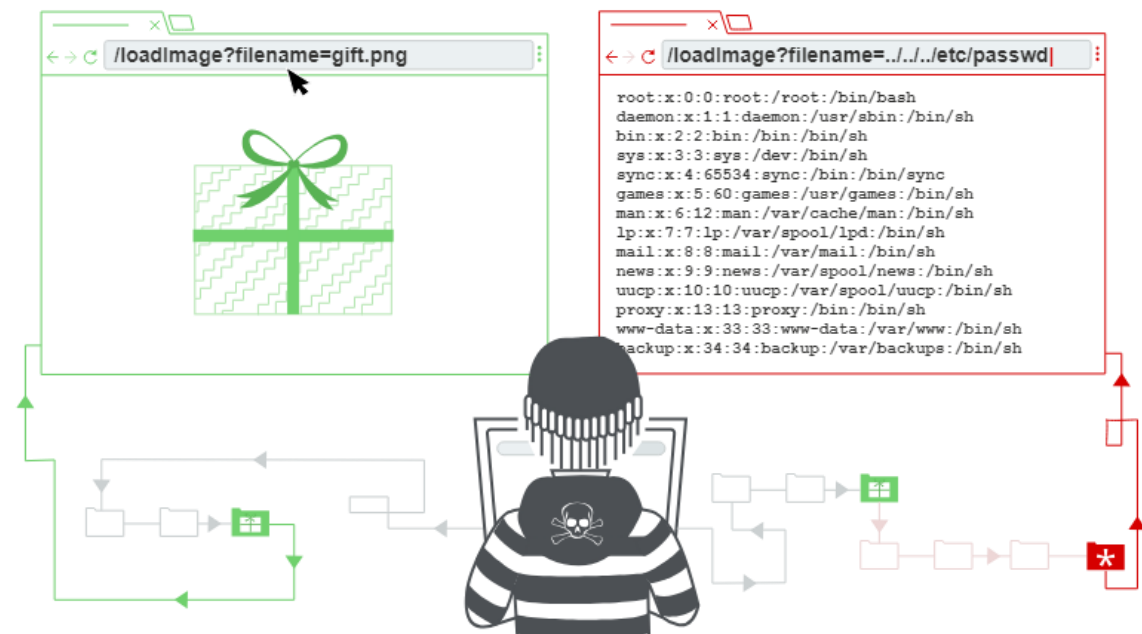
Mitigation

1. Check file extension – Web framework
2. Check the MIME type against a whitelist – Web framework
3. Use a library like “Python-magic” to deduce the file type based on signatures – Web framework
4. Limit the file size to prevent DOS attack – Web framework
5. Use an antivirus like ClamAV to check for virus infections – Web framework
6. Prevent file execution on server’s directory – Web server

Path traversal

Path traversal is also known as directory traversal. These vulnerabilities enable an attacker to read arbitrary files on the server that is running an application. This might include:

- Application code and data.
- Credentials for back-end systems.
- Sensitive operating system files.



Path traversal

- Imagine a shopping application that displays images of items for sale. This might load an image using the following HTML:
**
- The loadImage URL takes a filename parameter and returns the contents of the specified file. The image files are stored on disk in the location `/var/www/images/`. To return an image, the application appends the requested filename to this base directory and uses a filesystem API to read the contents of the file. In other words, the application reads from the following file path:
/var/www/images/218.png
- This application implements no defenses against path traversal attacks. As a result, an attacker can request the following URL to retrieve the `/etc/passwd` file from the server's filesystem:
https://insecure-website.com/loadImage?filename=../../etc/passwd

Path traversal

- This causes the application to read from the following file path:

/var/www/images/../../etc/passwd

- The sequence `../` is valid within a file path, and means to step up one level in the directory structure. The three consecutive `../` sequences step up from `/var/www/images/` to the filesystem root, and so the file that is actually read is:

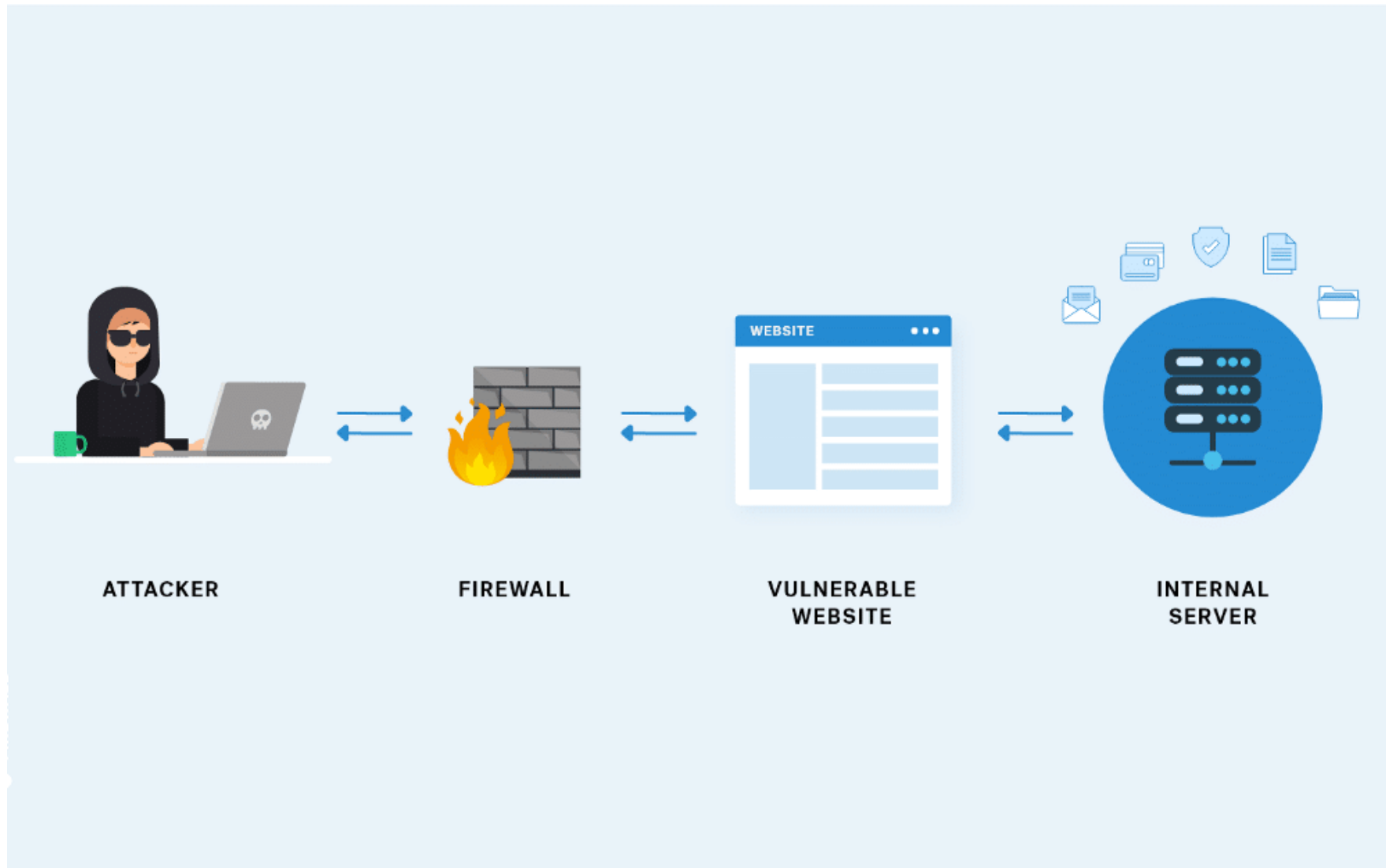
/etc/passwd

Server Side Request Forgery

SSRF

- Server-side request forgery is a web security vulnerability that allows an attacker to cause the server-side application to make requests to an unintended location.
- In a typical SSRF attack, the attacker might cause the server to make a connection to internal-only services within the organization's infrastructure. In other cases, they may be able to force the server to connect to arbitrary external systems. This could leak sensitive data, such as authorization credentials.

SSRF



SSRF example

- For example, imagine a shopping application that lets the user view whether an item is in stock in a particular store.
- To provide the stock information, the application must query various back-end REST APIs.
- It does this by passing the URL to the relevant back-end API endpoint via a front-end HTTP request.

SSRF attacks against the server

```
POST /product/stock HTTP/1.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 118

stockApi=http://stock.weliketoshop.net:8080/product/stock/check%3Fprc
```

This causes the server to make a request to the specified URL, retrieve the stock status, and return this to the user.

SSRF attacks against the server

```
POST /product/stock HTTP/1.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 118

stockApi=http://localhost/admin
```

- An attacker can visit the /admin URL, but the administrative functionality is normally only accessible to authenticated users.
- This means an attacker won't see anything of interest.
- However, if the request to the /admin URL comes from the local machine, the normal access controls are bypassed.
- The application grants full access to the administrative functionality, because the request appears to originate from a trusted location.

SSRF attacks against the server

- In an SSRF attack against the server, the attacker causes the application to make an HTTP request back to the server that is hosting the application, via its loopback network interface.
- This typically involves supplying a URL with a hostname like 127.0.0.1 or localhost.

SSRF attacks against the server

Why do applications behave in this way, and implicitly trust requests that come from the local machine? This can arise for various reasons:

- The access control check might be implemented in a different component that sits in front of the application server. When a connection is made back to the server, the check is bypassed.
- For disaster recovery purposes, the application might allow administrative access without logging in, to any user coming from the local machine. This provides a way for an administrator to recover the system if they lose their credentials. This assumes that only a fully trusted user would come directly from the server.
- The administrative interface might listen on a different port number to the main application, and might not be reachable directly by users.
- These kind of trust relationships, where requests originating from the local machine are handled differently than ordinary requests, often make SSRF into a critical vulnerability.

SSRF attacks against other back-end systems

- In some cases, the application server is able to interact with back-end systems that are not directly reachable by users.
- These systems often have non-routable private IP addresses.
- The back-end systems are normally protected by the network topology, so they often have a weaker security posture. In many cases, internal back-end systems contain sensitive functionality that can be accessed without authentication by anyone who is able to interact with the systems.
- In the previous example, imagine there is an administrative interface at the back-end URL `https://192.168.0.68/admin`.

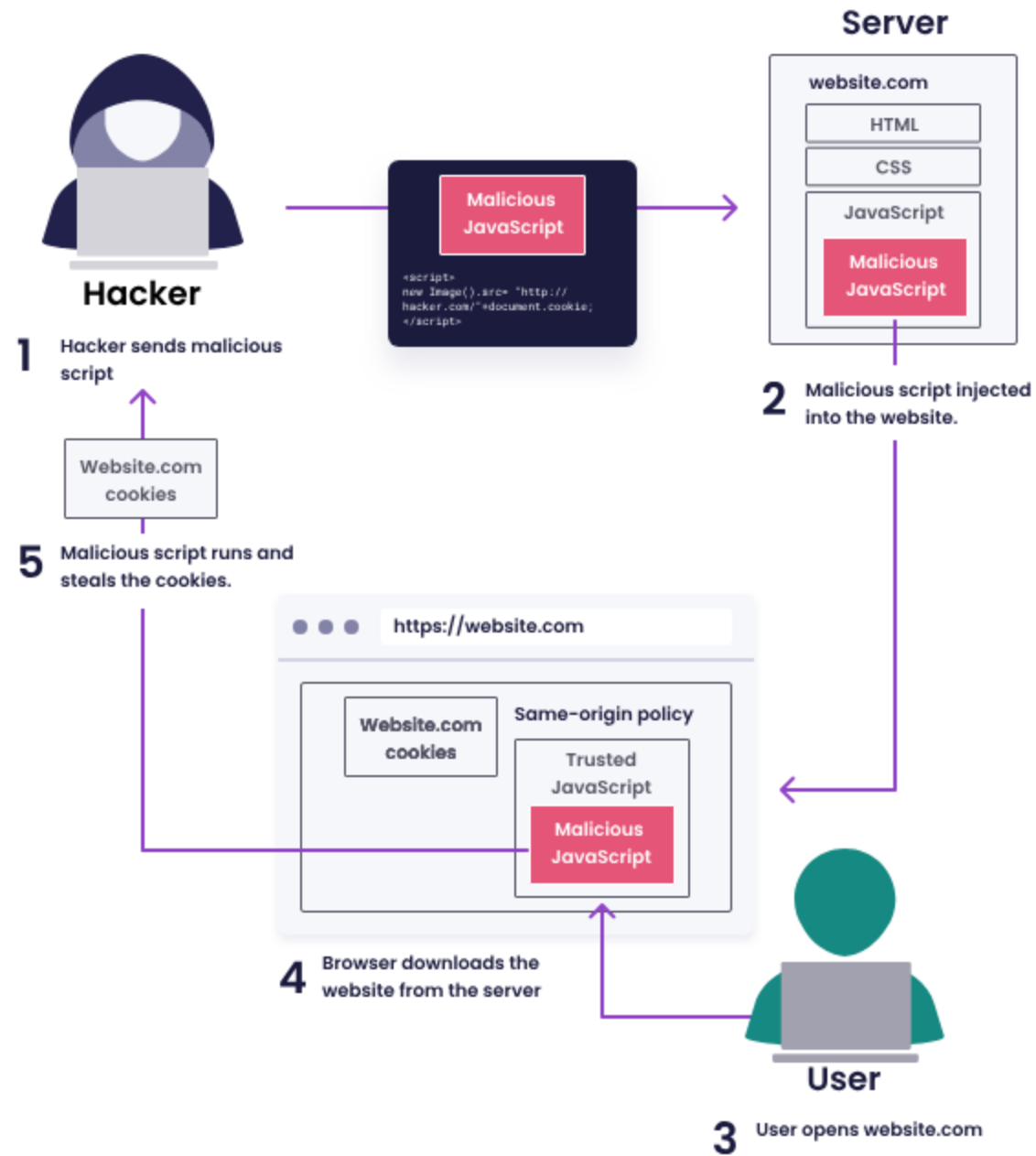
Cross Site Scripting

XSS

- Each time you visit a website, your browser downloads HTML, CSS, and JavaScript from the server that hosts the website. The browser interprets and displays HTML and CSS and executes JavaScript.
- When a piece of JavaScript is downloaded from a website, it can only access secrets (e.g. cookies) associated with that website. For instance, JavaScript code downloaded from startup.io cannot access cookies set by yourbank.com. If it could, it would be straightforward to steal sensitive information persisted by other websites, such as session tokens.

XSS

- Cross-site scripting (or XSS) is a code vulnerability that occurs when an attacker “injects” a malicious script into an otherwise trusted website.
- The injected script gets downloaded and executed by the end user’s browser when the user interacts with the compromised website. Since the script came from a trusted website, it cannot be distinguished from a legitimate script.



1/ Handling User Access

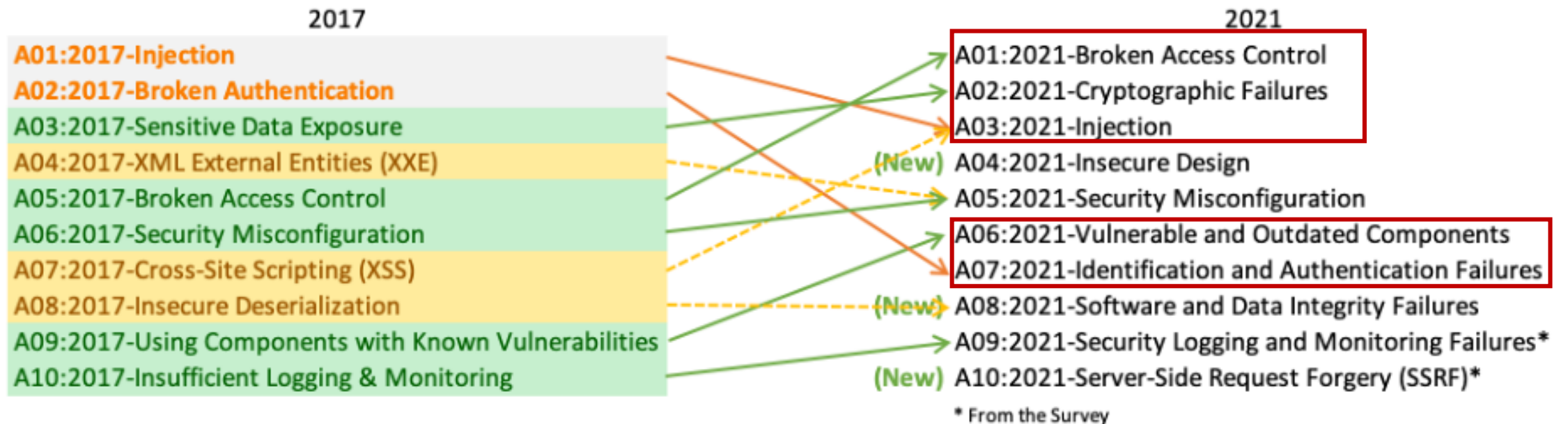
A central security requirement that virtually any application needs to meet is controlling users' access to its data and functionality. A typical situation has several different categories of user, such as anonymous users, ordinary authenticated users, and administrative users.

Furthermore, in many situations different users are permitted to access a different set of data. For example, users of a web mail application should be able to read their own e-mail but not other people's.

Most web applications handle access using a trio of interrelated security mechanisms:

- Authentication
- Session management
- Access control

OWASP Top 10



1/ Handling User Access - Authentication

The authentication mechanism is logically the most basic dependency in an application's handling of user access. Authenticating a user involves establishing that the user is in fact who he claims to be. Without this facility, the application would need to treat all users as anonymous — the lowest possible level of trust.

In security-critical applications such as those used by online banks, this basic model is usually supplemented by additional credentials and a multistage login process. When security requirements are higher still, other authentication models may be used, based on client certificates, smartcards, or challenge-response tokens.

In addition to the core login process, authentication mechanisms often employ a range of other supporting functionality, such as self-registration, account recovery, and a password change facility.

Despite their superficial simplicity, authentication mechanisms suffer from a wide range of defects in both design and implementation.

Common problems may enable an attacker to identify other users' usernames, guess their passwords, or bypass the login function by exploiting defects in its logic.

When you are attacking a web application, you should invest a significant amount of attention to the various authentication-related functions it contains. Surprisingly frequently, defects in this functionality enable you to gain unauthorized access to sensitive data and functionality.



What is session management

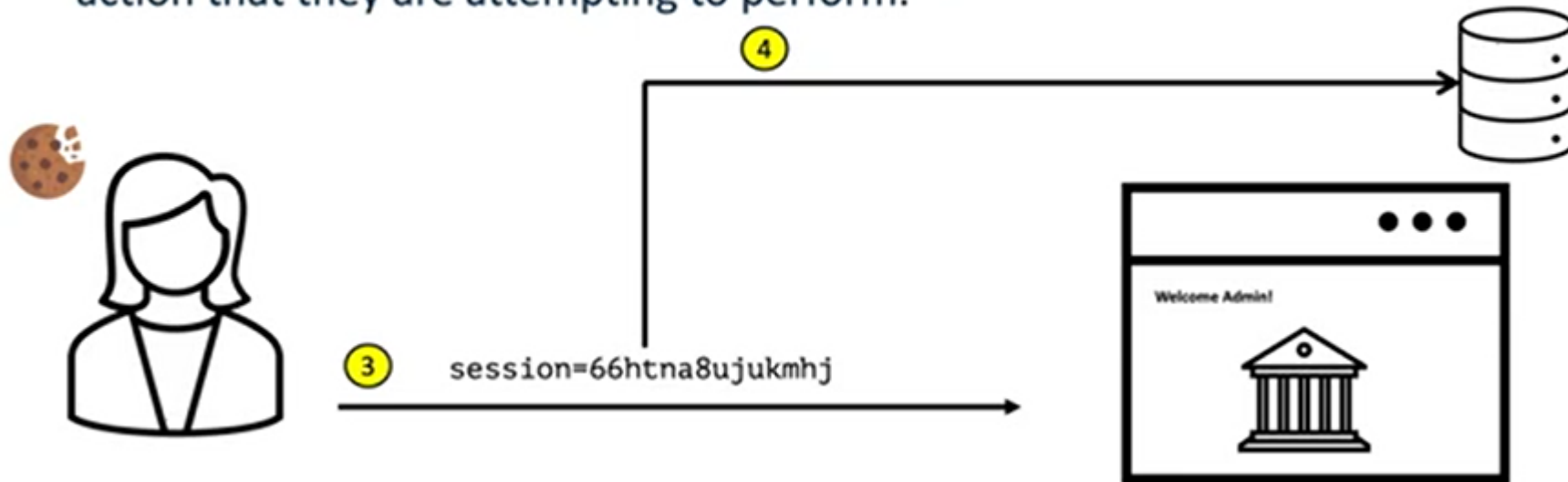
- **Session Management** identifies which subsequent HTTP requests are being made by each user.





What is Access Control

- **Access Control** determines whether the user is allowed to carry out the action that they are attempting to perform.

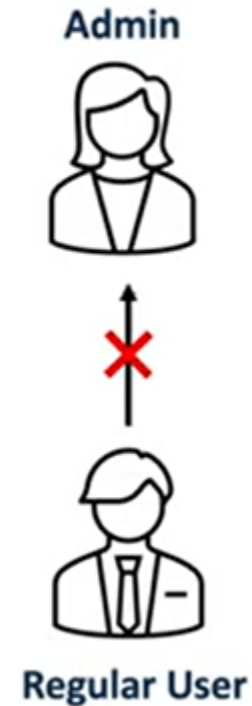




Types of Access Control

Three different types of access control:

- **Vertical Access Control** is used to restrict access to functions not available for other users in the organization.





Types of Access Control

Three different types of access control:

- **Horizontal Access Control** enables different users to access similar resource types.

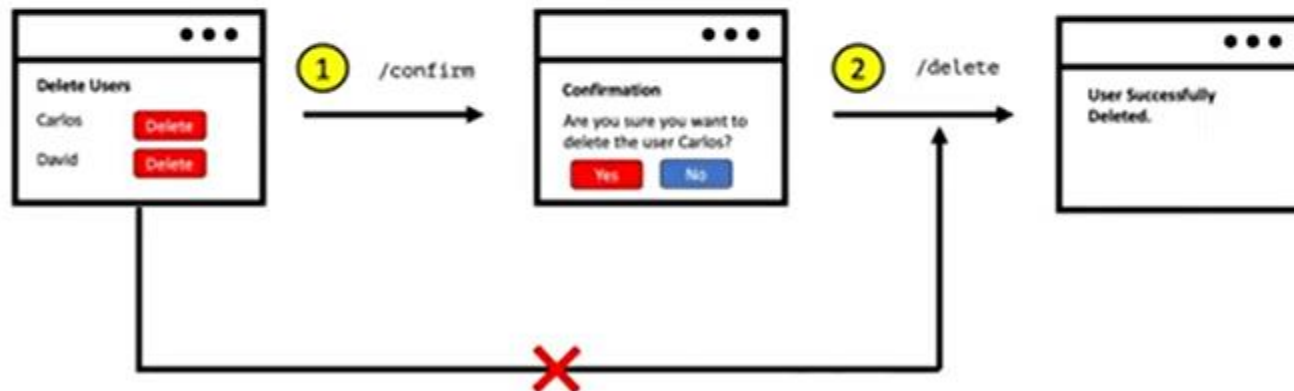




Types of Access Control

Three different types of access control:

- **Context-Dependent Access Control** restricts access to functionality and resources based on the state of the application or the user's interaction with it.





What is Broken Access Control

Broken Access Control vulnerabilities arise when users can act outside of their intended permissions. This typically leads to sensitive information disclosure, unauthorized access and modification or destruction of data.

1/ Handling User Access - Session Management

- After successfully logging in to the application, the user accesses various pages and functions, making a series of HTTP requests from his browser.
- At the same time, the application receives countless other requests from different users, some of whom are authenticated and some of whom are anonymous. To enforce effective access control, the application needs a way to identify and process the series of requests that originate from each unique user.
- Virtually all web applications meet this requirement by creating a session for each user and issuing the user a **token** that identifies the session.
- The session itself is a set of data structures held on the server that track the state of the user's interaction with the application. The token is a unique string that the application maps to the session.

1/ Handling User Access - Session Management

- When a user receives a token, the browser automatically submits it back to the server in each subsequent HTTP request, enabling the application to associate the request with that user.
- HTTP cookies are the standard method for transmitting session tokens, although many applications use hidden form fields or the URL query string for this purpose.
- If a user does not make a request for a certain amount of time, the session is ideally expired.

1/ Handling User Access - Session Management

- In terms of attack surface, the session management mechanism is highly dependent on the security of its tokens.
- The majority of attacks against it seek to compromise the tokens issued to other users.
- If this is possible, an attacker can masquerade as the victim user and use the application just as if he had actually authenticated as that user.
- The principal areas of vulnerability arise from defects in how tokens are generated, enabling an attacker to guess the tokens issued to other users,
- and defects in how tokens are subsequently handled, enabling an attacker to capture other users' tokens.



Cryptographic Failures



Cryptographic Failures

- A broad symptom rather than a root cause, the focus is on failures related to cryptography
- The first thing is to determine the protection needs of data in transit and at rest. For example, passwords, credit card numbers, health records, personal information, and business secrets require extra protection, mainly if that data falls under privacy laws, e.g., EU's General Data Protection Regulation (GDPR), or regulations.



Cryptographic Failures

- Security flaws that commonly lead to cryptography failures include:
- Transmitting secret data in plain text
- Use of old/less-secure algorithm
- Use of a hard-coded password in config files
- Improper cryptographic key management
- Insufficient randomness for cryptographic functions
- Missing encryption
- Insecure implementation of certificate validation
- Use of deprecated hash functions
- Presence of sensitive data in source control
- Use of passwords as crypto keys without a password-based key derivation function
- Exploitable side-channel information or cryptographic error messages



Cryptographic Failures - Examples

- Scenario #1: An application encrypts credit card numbers in a database using automatic database encryption. However, this data is automatically decrypted when retrieved, allowing a SQL injection flaw to retrieve credit card numbers in clear text.
- Scenario #2: A site doesn't use or enforce TLS for all pages or supports weak encryption. An attacker monitors network traffic (e.g., at an insecure wireless network), downgrades connections from HTTPS to HTTP, intercepts requests, and steals the user's session cookie. The attacker then replays this cookie and hijacks the user's (authenticated) session, accessing or modifying the user's private data. Instead of the above they could alter all transported data, e.g., the recipient of a money transfer.



OWASP Password Storage Cheat Sheet

- This cheat sheet advises you on the proper methods for storing passwords for authentication. When passwords are stored, they must be protected from an attacker even if the application or database is compromised.
- However, once an attacker has acquired stored password hashes, they are always able to brute force hashes offline. Defenders can slow down offline attacks by selecting hash algorithms that are as resource intensive as possible.



OWASP Password Storage Cheat Sheet

Usually, the attacker will repeat this process with a list of large number of potential candidate passwords, such as:

- Lists of passwords obtained from other compromised sites
- Brute force (trying every possible candidate)
- Dictionaries or wordlists of common passwords

While the number of permutations can be enormous, with high speed hardware (such as GPUs) and cloud services with many servers for rent, the cost to an attacker is relatively small to do successful password cracking, especially when best practices for hashing are not followed.



OWASP Password Storage Cheat Sheet

Methods for Enhancing Password Storage

Salting

A salt is a unique, randomly generated string that is added to **each** password as part of the hashing process. As the salt is unique for every user, an attacker has to crack hashes one at a time using the respective salt rather than calculating a hash once and comparing it against every stored hash. This makes cracking large numbers of hashes significantly harder, as the time required grows in direct proportion to the number of hashes.



OWASP Password Storage Cheat Sheet

Methods for Enhancing Password Storage

Using Work Factors

The work factor is the number of iterations of the hashing algorithm that are performed for each password (usually, it's actually 2^{work} iterations). The work factor is typically stored in the hash output. It makes calculating the hash more computationally expensive, which in turn reduces the speed and/or increases the cost for which an attacker can attempt to crack the password hash.

OWASP - Cryptographic Storage Cheat Sheet



- This article provides a simple model to follow when implementing solutions to protect data at rest.
- Passwords should not be stored using reversible encryption - secure password hashing algorithms should be used instead.

Where to Perform Encryption



Key Vault

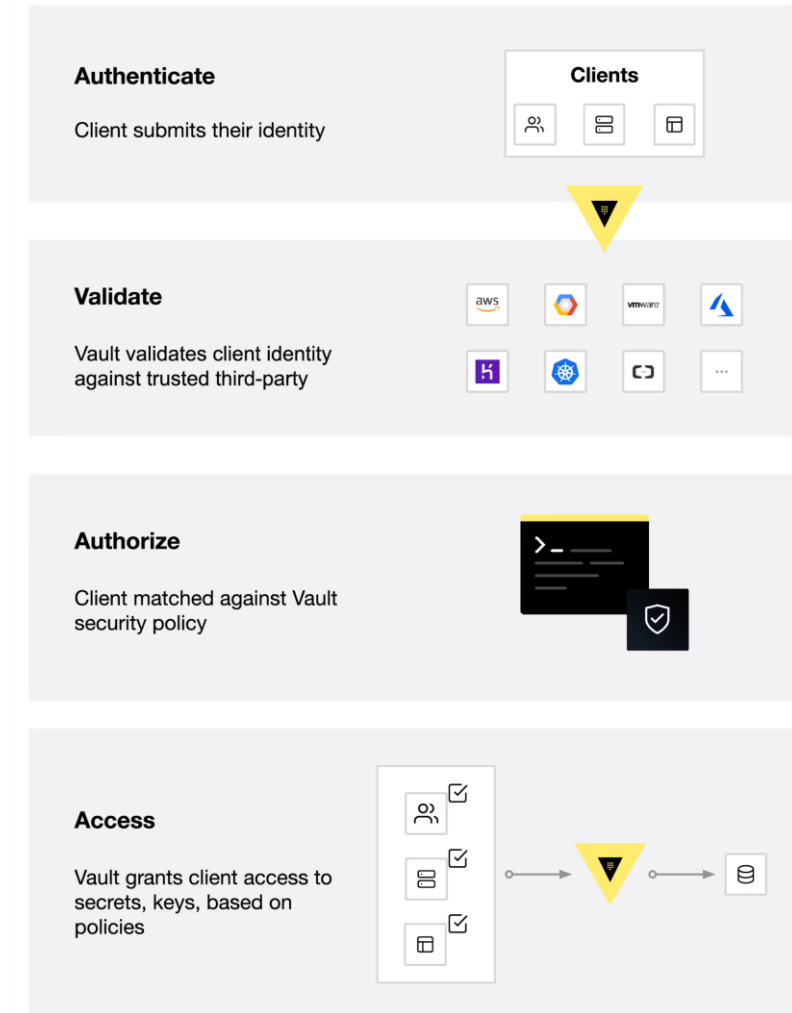
Vault is an identity-based secrets and encryption management system. A secret is anything that you want to tightly control access to, such as API encryption keys, passwords, and certificates. Vault provides encryption services that are gated by authentication and authorization methods.

Azure, AWS, GCP and Hashicorp are famous vault service providers



How does Vault work

Vault works primarily with tokens and a token is associated to the client's policy. Each policy is path-based and policy rules constrains the actions and accessibility to the paths for each client. With Vault, you can create tokens manually and assign them to your clients, or the clients can log in and obtain a token.





How does Vault work

- **Authenticate:** Authentication in Vault is the process by which a client supplies information that Vault uses to determine if they are who they say they are. Once the client is authenticated against an auth method, a token is generated and associated to a policy.
- **Validation:** Vault validates the client against third-party trusted sources, such as Github, LDAP, AppRole, and more.
- **Authorize:** A client is matched against the Vault security policy. This policy is a set of rules defining which API endpoints a client has access to with its Vault token. Policies provide a declarative way to grant or forbid access to certain paths and operations in Vault.
- **Access:** Vault grants access to secrets, keys, and encryption capabilities by issuing a token based on policies associated with the client's identity. The client can then use their Vault token for future operations.



TLS & SSL



Public, Private key and Certificates

What is TLS

TLS is based on SSL and was developed as a replacement in response to known vulnerabilities in SSLv3.

SSL is the term commonly used, and today usually refers to TLS.



Public, Private key and Certificates

SSL/TLS provides data encryption, data integrity and authentication.

This means that when using SSL/TLS you can be confident that

- No one has read your message
- No one has changed your message
- You are communicating with the intended person (server)

When sending a message between two parties you have two problems that you need to address.

- How do you know that no one has read the message?
- How do you know that no one has changed the message?



Public, Private key and Certificates

The solutions to these problems are to:

- Encrypt it.— This makes the content unreadable so that to anyone viewing the message it is just gibberish.
- Sign it— This allows the recipient to be confident that it was you who sent the message, and that the message hasn't been changed.



Public, Private key and Certificates

Almost all encryption methods in use today employ public and private keys.

These are considered much more secure than the old symmetrical key arrangement.

With a symmetrical key, a key is used to encrypt or sign the message, and the same key is used to decrypt the message.

With Public and Private keys, two keys are used that are mathematically related (they belong as a key pair), but are different.

This means a message encrypted with a public key cannot be decrypted with the same public key.

To decrypt the message you require the private key.



Public, Private key and Certificates

SSL/TLS use public and private key system for data encryption and data Integrity.

Public keys can be made available to anyone, hence the term public.

Because of this there is a question of trust, specifically:

How do you know that a particular public key belongs to the person/entity that it claims to be.

For example, you receive a key claiming to belong to your bank.

How do you know that it does belong to your bank?

The answer is to use a digital certificate.



Public, Private key and Certificates

A certificate serves the same purpose as a passport does in everyday life.

A passport established a link between a photo and a person, and that link has been verified by a trusted authority (passport office).

A digital certificate provides a link between a public key and an entity (business, domain name etc) that has been verified (signed) by a trusted third party (A certificate authority)

A digital certificate provides a convenient way of distributing trusted public encryption keys.

3/ Handling Attackers

A key function of the application's security mechanisms is being able to handle and react to these attacks in a controlled way. These mechanisms often incorporate a mix of defensive and offensive measures designed to frustrate an attacker as much as possible and give the application's owners appropriate notification and evidence of what has taken place.

Measures implemented to handle attackers typically include the following tasks:

- Handling errors
- Maintaining audit logs
- Alerting administrators
- Reacting to attacks

3/ Handling Attackers - Handling Errors

A key defense mechanism is for the application to handle unexpected errors gracefully, and either recover from them or present a suitable error message to the user. In a production context, the application should never return any system-generated messages or other debug information in its responses.

As you will see throughout this book, overly verbose error messages can greatly assist malicious users in furthering their attacks against the application. In some situations, an attacker can leverage defective error handling to retrieve sensitive information within the error messages themselves, providing a valuable channel for stealing data from the application.

Most web development languages provide good error-handling support through try-catch blocks and checked exceptions. Application code should make extensive use of these constructs to catch specific and general errors and handle them appropriately.

Furthermore, most application servers can be configured to deal with unhandled application errors in customized ways, such as by presenting an uninformative error message.

3/ Handling Attackers - Maintaining Audit Logs

Audit logs are valuable primarily when investigating intrusion attempts against an application. Following such an incident, effective audit logs should enable the application's owners to understand exactly what has taken place, which vulnerabilities (if any) were exploited, whether the attacker gained unauthorized access to data or performed any unauthorized actions, and, as far as possible, provide evidence of the intruder's identity.

In any application for which security is important, key events should be logged as a matter of course. At a minimum, these typically include the following:

- All events relating to the authentication functionality, such as successful and failed login, and change of password
- Key transactions, such as credit card payments and funds transfers
- Access attempts that are blocked by the access control mechanisms
- Any requests containing known attack strings that indicate overtly malicious intentions